BET 2026 ENSEMBLE · OBJECTIVE DIAGNOSTICS

What the reporting-rate flag actually does to the tag penalty

The `RR (tag reporting)` grid axis moves the tagged-fish reporting-rate penalty by a median of roughly 38 units. This traces that gap to its source in the MFCL objective, tests whether M0 explains it, and asks whether the **include** arm's low penalties are earned or censored.

80 retained PARs · 88 retained models · 44 of 48 checks held

MAX RESIDUAL	ADJUSTED ARM EFFECT	RAW MEDIANS	CHECKS
4.9×10^{-10}	+35.5	63.2 / 101.5	44 / 4
reconstruction vs reported, all 80 members · tolerance 1×10^{-6}	exclude − include, 95% CI [28.7, 42.4]	include (n=41) vs exclude (n=47)	held / did not hold all four listed below

Every directional claim below is followed by the named check that backs it. Checks that did not hold are shown in the same form, not paraphrased away — three of the four are results rather than defects.

ARITHMETIC · TASK 3 · GATES EVERYTHING DOWNSTREAM

The penalty is reproduced exactly

The formula is transcribed from `multifan-cl` `src/callpen.cpp`, not inferred. The branch taken is the one guarded by `age_flags(198) != 0`, which is 1 in all 80 PARs:

```
for i in 1..num_tag_releases+1, j in 1..num_fisheries:
    if gflag[group(i,j)] == 0 and tag_fish_rep_penalty(i,j) > 0:
        gflag[group(i,j)] = 1
        xy += tag_fish_rep_penalty(i,j)
              * square(tag_fish_rep(i,j) - tag_fish_rep_target(i,j)/100)
```

Weight multiplies, scale is proportion, and the sum runs over unique group ids — each taken at the first cell in row-major order with a positive penalty.

HELD `task3.reconstruction-matches-reported`

max|resid| = 4.86e-10 (tol 1e-06); median resid = 2.32e-12

Three structural corrections

- **The selector is the penalty matrix, not the active flags.** `tag_fish_rep_active_flags` governs which rates are *estimated*; `tag_fish_rep_penalty` governs which carry a prior. Here the two sets coincide, so nothing changes numerically — but the code must select on the penalty.
- **Five groups carry an informative prior, not three.** There are three distinct priors, each shared between an RTTP group and a PTTP/pooled group.
- **Group 18 is `PS.EAST.3`, not region 4.** The three priors are region-2 PS, region-3-west PS and region-3-east PS.

A stop condition fired, and it is not one

`tag_flags(:,2)` is **not constant within a PAR** in 26 of 80 members. It is a per-release-group flag, and the design deliberately forces face-value removal on release groups whose mixing window is zero — **include** is undefined when there is no window to raise. The carve-out is exact, and the arm label read off the groups that *do* have a window agrees with the design table for all 80.

HELD `task1.flag2-exceptions-are-exactly-zero-mixing-groups`

under `include`, tag_flags(:,2)=1 iff tag_flags(:,1)=0

HELD `task1.arm-label-agrees-with-design`

The stop condition needs restating as *"not constant across release groups with a mixing window"*.

EMPIRICAL · TASK 4 · THE CONFOUND CHECK

M0 does not explain the gap

M0 has a far tighter relationship with this penalty than the flag axis does, and these are selected models rather than a fully crossed grid — so the first question is whether M0 is leaking in. It is not. The standardised mean difference between arms is **−0.076**, and adding M0 to the model *raises* the arm coefficient from 33.98 to 35.54 while R^2 climbs from 0.454 to 0.852. M0 is a strong predictor (≈ 1558 per unit quarterly M, $p \approx 1 \times 10^{-24}$) but orthogonal to the arm: it explains scatter, not the gap.

HELD `task4.M0-balanced-across-arms`

standardised mean difference (exclude - include) = -0.076

HELD `task4.arm-effect-robust-to-adding-M0`

arm coef 33.98 -> 35.54 on adding M0 (+4.6%)

HELD `task4.adjusted-arm-effect-positive`

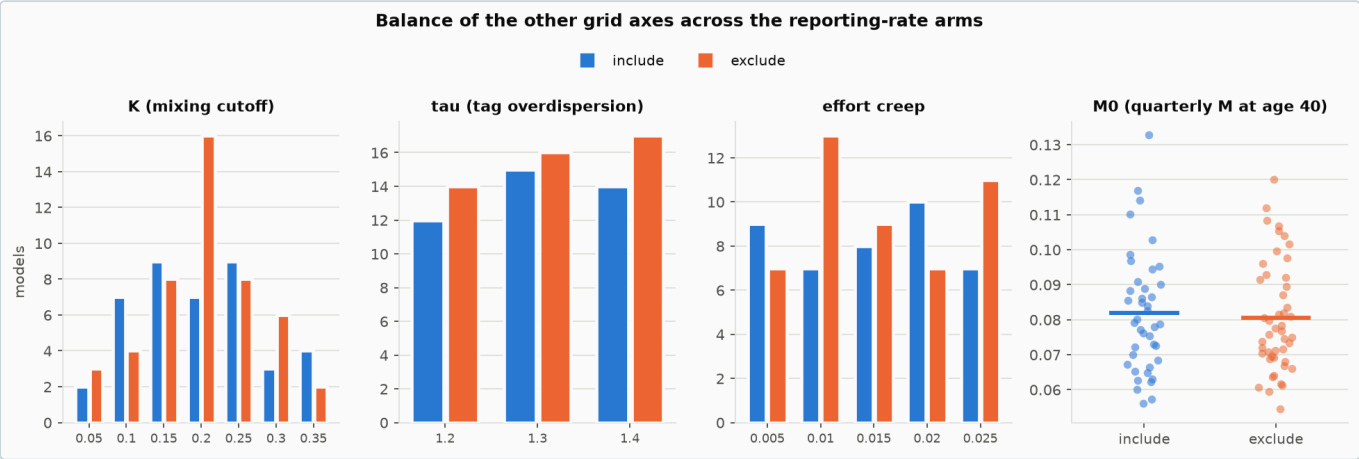
adjusted arm effect (exclude - include) = 35.54 [28.72, 42.36]

Quote the adjusted estimate, not the violin medians. On the log scale the same fit gives +0.526 [0.430, 0.622] — the **exclude** arm carries about **1.69×** the penalty. Treating tau, K and effort creep as factors instead of continuous moves the estimate to +36.5, so the scale choice does not matter.

One balance check did not hold

> **DID NOT HOLD** `task4.K-balanced-across-arms`
>
> largest level-share difference = 0.170

K is materially imbalanced at K = 0.20 — include 7/41, exclude 16/47. K is in every adjusted model and the adjusted estimate is larger than the unadjusted one, so the imbalance does not manufacture the effect. It is reported because it is real.



Balance of the other grid axes across the arms. M0 (right) overlaps almost completely — the candidate confound is not one. K (left) is the axis that does not balance.

EMPIRICAL · TASK 5 · DIRECTION AND MAGNITUDE

Larger excursions under exclude, and upward in both arms

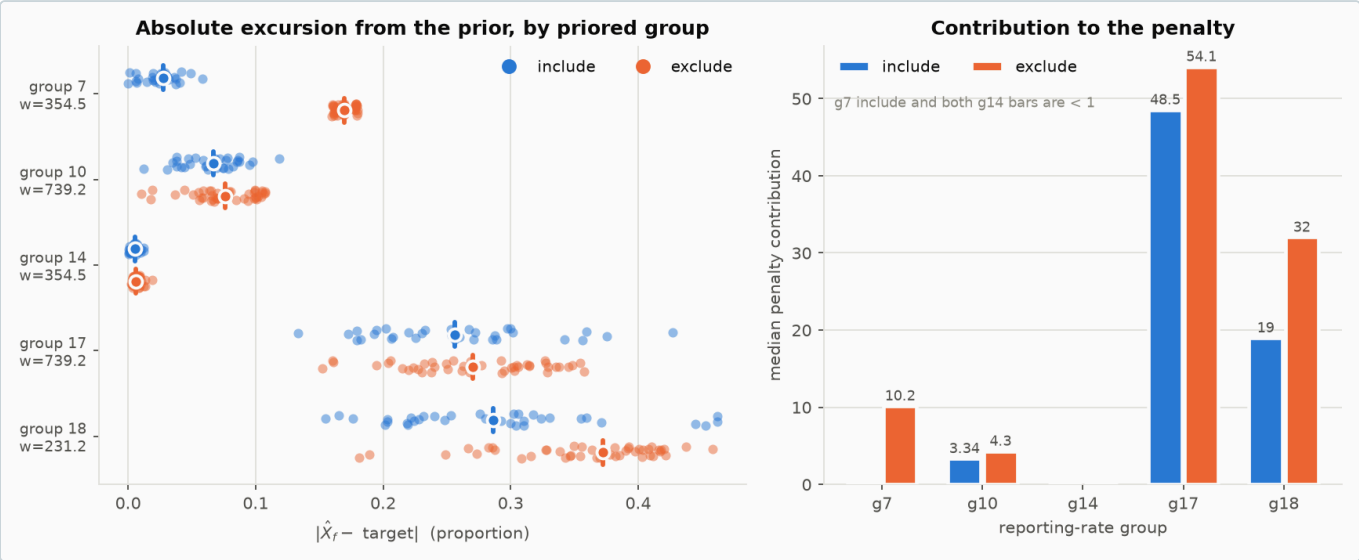
The prediction is about **magnitude, not sign**. It holds for every priored group, and the deviation is *upward* in both arms — nothing here supports a downward push.

GROUP	PRIOR	WEIGHT	|DEV| INCLUDE	|DEV| EXCLUDE	DIFF	MWU P
7 · RTTP PS.2	0.4962	354.5	0.0276	0.1693	**+0.1417**	2.8e−14
10 · RTTP PS.WEST.3	0.5121	739.2	0.0672	0.0763	+0.0091	0.028
14 · PTTP PS.2	0.4962	354.5	0.0054	0.0061	+0.0007	0.079
17 · PTTP PS.WEST.3	0.5121	739.2	0.2562	0.2706	+0.0144	0.146
18 · PTTP PS.EAST.3	0.5282	231.2	0.2864	0.3722	**+0.0857**	5.3e−05

> **HELD** `task5.exclude-has-larger-abs-excursion-pooled-priored`
>
> median |dev| include 0.06721 vs exclude 0.1687

HELD `task5.sign-preserved-across-arms-g18`

median signed dev include +0.2864, exclude +0.3722



Absolute excursion from the prior (left) and penalty contribution (right) for the five priored groups. Group 7 is the clearest single signal: its median estimate moves from 0.524 — essentially at its 0.496 prior — to 0.666.

The arm gap is not group 17

DID NOT HOLD `task5.arm-gap-dominated-by-group-17`

group 17 supplies 19.3% of the median total-penalty arm gap; group 18 supplies 45.0% and group 7 34.1%

This is a result, not a nuisance. Group 17 dominates the *level* of the penalty — 68% of the include-arm median, 53% of the exclude-arm median — but supplies only 19% of the arm *gap*. The gap comes from group 18 and group 7. Level and gap are answered by different parameters, and the brief's expectation was built on the level.

DID NOT HOLD `task5.weight-1-share-under-2pc-in-every-member`

max share of total penalty from weight-1 groups = 0.03635 (median 0.007791, 90th pct 0.01787)

The weight-1 groups contribute a median 0.78% of the total penalty but up to 3.6% in the worst member — under 5% everywhere, so the tight screen fails while the substance holds. The story is about five parameters, and effectively about three.

EMPIRICAL · TASK 6 · THE MECHANISM

The gap scales with the mixing window

The realised mixing configuration does what the axis label implies: median release-weighted mean mixing period falls from 3.53 periods at K = 0.05 to 0.84 at K = 0.35, and the count of zero-mixing release groups rises from 0 to 18.

HELD `task6.mixing-window-decreases-monotonically-with-K`

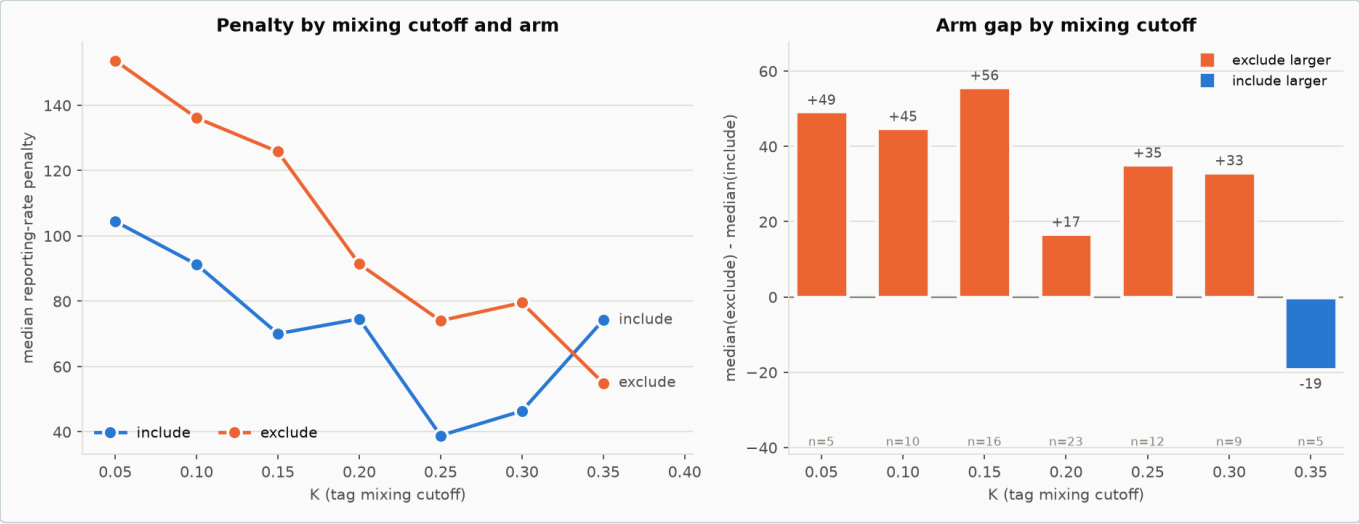
median release-weighted mean mixing period by K is monotone decreasing; Spearman(K, mean mixing) = -1.0000

HELD `task6.arm-gap-shrinks-with-K`

arm x K interaction on the raw penalty = -218 [-298.3, -137.7], p=7.74e-07

HELD `task6.arm-gap-grows-with-mean-mixing-period`

arm x mean-mixing interaction = 21.11 [13.65, 28.56], p=3.09e-07



Median penalty by mixing cutoff (left) and the arm gap at each level (right). The gap runs +49 at K = 0.05 down to −19 at K = 0.35, where only five models sit.

Confirmed in the source, not just inferred

`src/tag3.cpp` switches on `tag_flags(it,2)` **only inside the mixing window**:

```
tag_flags(it,2)==0  →  actual_tag_catch = tot_tag_catch / (1e-6 + tag_rep_rate)   [include]
tag_flags(it,2)==1  →  actual_tag_catch = tot_tag_catch                           [exclude]
```

Under **include** the mixing-period removal falls as `X_f` rises, so the surviving cohort — and with it the predicted post-mixing returns — increases with `X_f` on top of the direct scaling. That is the second lever, and it exists only where a mixing window exists. No window, no lever: exactly the K = 0.35 end where the gap vanishes.

EMPIRICAL · TASK 7 · WHERE THE MECHANISM STOPS

ψ̂ does not order the effect across groups

Across the five priored groups, Spearman's rho between in-window recapture share and the arm difference in median |dev| is **+0.10**; across all twelve penalised groups it is **−0.13**. The sign check passes, but at n = 5 it carries almost no information and should not be read as support.

HELD `task7.highest-psi-group-shows-larger-excursion-difference-than-lowest`

highest-psi group g7 (psi=0.994) d|dev|=+0.1417; lowest-psi group g17 (psi=0.349) d|dev|=+0.0144

The extremes line up — highest $\hat{\psi}$ (group 7, 0.994) has the largest excursion difference, lowest $\hat{\psi}$ (group 17, 0.349) nearly the smallest — but the middle does not. Group 18 ($\hat{\psi}$ = 0.406) shows +0.086 while group 14 ($\hat{\psi}$ = 0.833) shows +0.0007. Group 14 has **6** recaptured fish against group 18's **3117**, so recapture volume, not in-window share, is what separates them.

Read this as: the mechanism holds at the member level and does not distribute across groups in proportion to their in-window share. Reported descriptively; no model is fitted to n = 5.

EMPIRICAL · TASK 8 · CENSORING SCREENS

Both screens cut against include

Reporting-rate bound

`parest_flags(33) = 99` in every member, so `tag_fish_rep` is bounded at **0.99**.

DID NOT HOLD `task8.include-arm-not-more-often-at-the-rr-bound`

priored-group bound hits: include 5.9% vs exclude 0.0% of members

Two of 34 **include** members have a priored group pinned within 1e−3 of the bound — both group 18 at 0.990 — against none of the 46 **exclude** members. Small, but one-directional: part of the include arm's low tail is truncation rather than fit.

Survival floor

`src/tag3.cpp` floors survival with `posfun` once `surv_rate = 1 − raised_removal / tags_present ≤ 0.22`. Because the raised removal is `reported / X_f`, only the **include** arm can drive it.

HELD `task8.floor-engagement-higher-under-include`

share of members with >=1 engaging release group: include 100.0% vs exclude 0.0%

HELD `task8.lowest-penalty-include-members-are-more-floor-engaging`

lowest-penalty tercile 2.00 vs highest 1.36 engaging release groups; Spearman(penalty, n_engaging) = -0.588

Minimum proxy survival reaches **−2.98** under include against 0.42 under exclude. Release groups flagged: **98** (the pooled group, in all 34 include members), **21** (14 members), 15 (6) and 5 (1).

HELD `task8.discarded-release-groups-appear-in-floor-screen`

of the groups whose posfun penalty MFCL discards (21, 72, 112), the screen flags [21]; 4 distinct release groups flagged overall

MFCL **discards the posfun penalty outright** for release groups 21, 72 and 112 — `switch (it) { case 72: case 21: case 112: break; default: _ffpen+=ffpen; }` — so 14 of the 55 member × release-group flags accrue no penalty at all.

The floor screen is a `bet.tag`-only proxy, deliberately conservative: it ignores natural mortality, tag shedding and the per-region evaluation, all of which would make engagement more likely. A flagged group is a strong candidate; an unflagged one is not cleared.

INTERPRETATION

What this does and does not license

The flag axis does real, localised work

Adjusted effect +35.5 [28.7, 42.4] penalty units, robust to M0, scaling with the mixing window exactly as an extra-lever account predicts.

A lower penalty is not a better model

The **include** arm's extra lever is an accounting identity — the same observations differentiated twice — not new information. Both censoring screens then push the same way: include is the only arm that can hit the survival floor, it does so in every member, it does so most in its own lowest-penalty members, and part of that floor penalty is discarded outright for release group 21. Its small penalties are partly cushioned rather than earned. This component is not evidence for **include**.

The mechanism is confirmed at member level, not group level

Task 6 passes decisively; Task 7 does not. Any account of *why* particular reporting-rate groups move needs something beyond in-window share — recapture volume is the obvious candidate, and is untested here.

Not tested

Whether the arm difference in this component propagates to management quantities. Nothing here speaks to that.

Analysis code: `analysis/rr-penalty/`, reproduced by `analysis/rr-penalty/run-all.sh`. Tidy CSVs, figures and the full check log in `out/`. Objective components and the design table cover all 88 retained models; only 80 have a retained `final.par`, so Tasks 5–8 run on those 80 — the arm effect is 35.5 on 88 and 34.3 on 80.